

HelixSpecifier

HelixSpecifier

Développement piloté par les spécifications, dont le cérémonial s'adapte à l'ampleur du travail.

Résumé

HelixSpecifier est un moteur Go qui fusionne trois méthodologies de développement — le flux piloté par les spécifications de SpecKit, la rigueur TDD de Superpowers et le cycle de vie par jalons de GSD — en un processus adaptatif unique. Il classe chaque tâche en fonction de son niveau d'effort et ajuste en conséquence l'ampleur des processus appliqués.

Description courte

HelixSpecifier est un moteur de fusion pour le développement piloté par les spécifications, conçu pour les agents AI. Il intègre SpecKit, Superpowers et GSD, classe les tâches selon leur niveau d'effort, exécute des phases de spécification étayées par des débats, impose un ratio minimal de tests par rapport au code, et tire des enseignements de chaque flux achevé.

Description longue

HelixSpecifier est un moteur de développement piloté par les spécifications (SDD) écrit en Go (module `digital.vasic.helixspecifier`), conçu comme un composant de l'ensemble HelixAgent AI. Il prend trois pratiques de développement habituellement dissociées — tant dans les outils que dans les mentalités — et les fusionne en un flux de travail adaptatif : le processus SDD en sept phases de SpecKit (*Constitution, Spécifier, Clarifier, Planifier, Tâches, Analyser, Implémenter*), la discipline TDD de Superpowers avec exécution parallèle par sous-agents, et la gestion du cycle de vie par jalons

de GSD. Chaque pilier conserve sa force propre ; c'est le moteur qui les fait fonctionner comme un flux cohérent plutôt que comme un pipeline assemblé manuellement.

Son principe central est celui du *cérémonial adaptatif* : le moteur classe les tâches entrantes selon leur niveau d'effort et ajuste l'ampleur des processus en conséquence, évitant ainsi qu'une correction d'une ligne ne soit soumise au même rituel lourd qu'une fonctionnalité majeure — et qu'une fonctionnalité majeure ne soit traitée avec la rigueur d'une simple coquille. Dix fonctionnalités clés s'articulent autour de cette colonne vertébrale : l'exécution parallèle des tâches avec concurrence limitée, un « Constitution as Code » lisible par machine imposant automatiquement des règles obligatoires, le « Nyquist TDD » qui suit et impose un ratio minimal entre tests et implémentation, des débats multi-agents et multi-tours pour affiner les spécifications, l'apprentissage adaptatif des compétences, l'analyse des bases de code existantes, la spécification prédictive à partir de motifs historiques, le transfert de connaissances entre projets, l'ajustement dynamique du cérémonial en cours d'exécution, et une mémoire persistante des spécifications dotée d'une recherche sémantique.

Il s'utilise comme un module Go — via `go get` ou une directive `replaces` locale — derrière une interface moteur API délibérément minimaliste : il suffit d'enregistrer les trois piliers ainsi qu'un ajusteur de cérémonial et une mémoire de spécifications, de classer l'effort requis par la tâche, puis d'exécuter le flux complet pour obtenir un résultat noté en termes de qualité. La surface est simple ; l'orchestration sous-jacente ne l'est pas. À l'instar du reste de la famille Helix, il est développé sous un régime de vérification anti-bluffer, avec un exécuteur de défis intégré testant du code réel plutôt que des simulations.

Pourquoi nous l'avons créé

Le développement piloté par les spécifications, le TDD rigoureux et la gestion par jalons sont généralement trois pratiques distinctes, chacune associée à un outil dédié. HelixSpécifier a été conçu pour permettre à un agent AI (HelixAgent) d'exécuter ces trois approches comme un flux cohérent et auto-adaptatif, plutôt que de devoir les assembler manuellement.

Contenu

Pourquoi c'est révolutionnaire

Il rend le processus proportionnel au travail — automatiquement. Les équipes se retrouvent généralement bloquées à l'un de ces deux extrêmes peu satisfaisants : une lourdeur procédurale systématique (sécurisant mais lent, et discrètement mal vécu) ou l'absence totale de formalisme (rapide... jusqu'à ce que ça ne le soit plus). HelixSpécifier élimine ce compromis en adaptant le niveau de formalité à l'effort classé de chaque tâche, et en le réajustant en temps réel à mesure que le travail se précise. Une capacité jusqu'alors irréalisable : un processus qui se dimensionne automatiquement pour chaque tâche — et, en plus, des décisions de spécification étayées par un débat multi-tours et multi-agents, avec notation des positions, plutôt que par la première intuition d'un seul intervenant.

Ce qui est innovant

- **Cérémonial adaptatif** — le niveau de processus est piloté par des métriques de qualité en temps réel et ajusté dynamiquement, sans être figé à l'avance.
- **TDD Nyquist** — un seuil de ratio tests/implementation (minimum 2x), s'inspirant du théorème d'échantillonnage de Nyquist : pour capturer fidèlement un comportement, il faut l'échantillonner bien au-delà de sa fréquence, donc les tests doivent couvrir le code avec une marge suffisante.
- **Architecture de débat** — affinement collaboratif des spécifications en plusieurs tours, impliquant plusieurs agents, où les positions sont proposées, notées et convergées, remplaçant une opinion unique par un processus contradictoire.
- **Spécification prédictive et transfert inter-projets** — le moteur exploite les flux accumulés pour anticiper les spécifications et transposer les connaissances durement acquises d'un projet à l'autre.
- **Constitution sous forme de code** — les règles de projet, rendues lisibles par machine, sont appliquées par le moteur, et non laissées à la vigilance des relecteurs.

Principaux défis techniques et solutions apportées

- **Fusionner trois méthodologies sans qu'elles ne s'opposent** — SpecKit, Superpowers et GSD supposent chacune maîtriser le flux de travail. Résolu grâce à un moteur de fusion qui intègre chaque pilier derrière une interface commune et les fait converger vers un cycle de vie unifié, les

transformant en un processus cohérent plutôt qu'en trois approches concurrentes.

- **Déterminer le niveau de processus réellement nécessaire pour une tâche donnée** — une estimation trop élevée ralentit tout ; trop basse, et le travail risqué est livré sans contrôle. Résolu par un classificateur d'effort qui évalue la charge, alimentant un module d'ajustement dynamique du cérémonial à mesure que l'exécution progresse.
- **Maintenir une qualité élevée des spécifications sans gardien humain pour chaque décision** — résolu en remplaçant les spécifications ponctuelles par un affinement collaboratif, où des agents notent les positions concurrentes sur plusieurs tours, et en imposant les ratios TDD Nyquist pour empêcher l'implémentation de devancer ses tests.

Pile technologique

- **Go** — choisi pour que le moteur soit livré sous forme d'un binaire importable unique, sans dépendances runtime ; son modèle de concurrence permet une répartition parallèle bornée des tâches et des tours de débat multi-agents sans complexité de gestion des threads.
- **logrus** — journalisation structurée, intégrée au moteur et aux trois piliers, rendant lisibles a posteriori les décisions d'un flux (classification, ajustements du cérémonial, résultats des débats).
- **Pilier SpecKit** — le processus de développement piloté par spécifications en sept phases (Constitution → Spécifier → Clarifier → Planifier → Tâches → Analyser → Implémenter), offrant l'épine dorsale disciplinée de la transformation d'une spécification en code.
- **Pilier Superpowers** — rigueur du TDD avec exécution parallèle de sous-agents, garantissant une approche *test-first* et une parallélisation qui maintient l'implémentation à la fois honnête et rapide.
- **Pilier GSD** — gestion des jalons et du cycle de vie, conférant au flux sa notion d'« achevé » et son avancée par étapes.
- **Mémoire des spécifications** — un index persistant et sémantiquement interrogeable des spécifications passées, substrat rendant possible la spécification prédictive et le transfert inter-projets, évitant de repartir de zéro à chaque fois.

Contenu

Notes sur le statut et l'honnêteté

- **Statut : bêta.** Consommé en tant que composant module Go de HelixAgent.

- **Licence : à déterminer.** Aucune LICENCE n'a été détectée via le GitHub API — NON VÉRIFIÉ / non déclarée.
- Le nom d'affichage « HelixSpecifier » correspond au dépôt specifier.

Niveau de priorité : Helix-primaire.